

## CSE422: Artificial Intelligence LAB MID EXAM

TOTAL MARKS: 8

DURATION: 60 MINUTES

*Note: You may create arbitrary input and output cases to test your designed algorithms.*

---

### Problem 1: Alpha-Beta Pruning [3.5 Marks]

**Scenario:** You are hired by a mobile gaming studio to build the AI opponent for their new two-player strategy game, *Coin Duel*. Every possible sequence of moves from the current position can be laid out as a game tree, where the root is the current board state, internal nodes alternate between your turn (MAX, trying to maximize the final score) and the opponent's turn (MIN, trying to minimize it), and each leaf represents a possible ending of the game with a precomputed utility score (the point margin in your favor). The studio's engineers are skeptical that pruning actually saves any work on their specific game tree, so besides returning the optimal value, they have also asked you to report exactly **which branches your algorithm skipped**, so they can verify the savings themselves.

**Task:** Implement a function that computes the optimal utility value achievable from a given node using Alpha-Beta Pruning, while also recording the name of every branch that gets skipped due to pruning along the way.

**Inputs may include:**

- **tree:** A representation of the game tree. You may use any data structure you find convenient. For example, a dictionary mapping each node's name to either an integer (if it is a leaf, representing its terminal utility score) or a list of its children's names in left-to-right exploration order (if it is an internal node). You are not required to use a dictionary; any equivalent representation is acceptable.
- **node:** The name of the node currently being evaluated, starting with the root (e.g., 'A').
- **alpha, beta:** The current best values that MAX and MIN can guarantee so far along the path to the root (initially  $-\infty$  and  $+\infty$ , respectively).
- **maximizing\_player:** A boolean indicating whose turn is being evaluated at node; True if it is a MAX node (your turn), False if it is a MIN node (the opponent's turn).

**Output:** Your function should produce two things:

1. An integer representing the optimal (minimax) value of the root node. [1.5 Marks]
2. A list `pruned_nodes` containing the names of every branch that was cut off during the search and therefore never evaluated. [2 Marks]

**Sample Input:**

```

tree = {
  'A': ['B', 'C'],
  'B': ['D', 'E'],
  'C': ['F', 'G'],
  'D': ['D1', 'D2'],
  'E': ['E1', 'E2'],
  'F': ['F1', 'F2'],
  'G': ['G1', 'G2'],
  'D1': 3, 'D2': 5,
  'E1': 6, 'E2': 9,
  'F1': 1, 'F2': 2,
  'G1': 0, 'G2': -1
}

```

### Sample Output:

```

Optimal value: 5
pruned_nodes: ['E2', 'G']

```

## Problem 2: Genetic Algorithm [4.5 Marks]

**Scenario:** You are a portfolio manager tasked with assembling an optimal set of investments. Given a pool of available stocks, each with a specific expected return and purchase cost, your goal is to maximize the portfolio's total return. A chromosome is represented as a binary string, where '1' means a stock is included in the portfolio and '0' means it is excluded.

**Task:** You must implement the following four helper functions and combine them inside a main `generate_evolution` function to simulate exactly 8 generations of a Genetic Algorithm.

1. `fitness(chromosome, returns, costs, budget)`: Calculates the fitness of a portfolio based on the selected stocks. If the total purchase cost exceeds the budget, a penalty is applied:  

$$Fitness = \max(0, Total\ Return - 3 \times (Total\ Cost - Budget))$$
2. `select_parents(population, fitness_scores)`: Performs truncation selection. Sorts the population by fitness in descending order and returns the fittest 3/4 of the population as the mating pool.
3. `crossover(parent1, parent2)`: Performs single-point crossover. Chooses a random crossover point along the chromosome and swaps the segments beyond that point between the two parents to produce two offspring.
4. `mutate(chromosome)`: Randomly flips exactly one bit ( $0 \rightarrow 1$  or  $1 \rightarrow 0$ ) in the given chromosome.

5. `generate_evolution(population, returns, costs, budget)`: Uses the above four functions sequentially. For exactly 8 iterations (generations): evaluate the population's fitness, apply truncation selection to obtain the fittest 3/4 as the mating pool, randomly pair individuals from this mating pool and apply single-point crossover until enough offspring are produced to replace the old population, then mutate the offspring. Return the final population.

**Inputs for `generate_evolution`:**

- `population`: A list of binary strings (e.g., ['1101', '0110', '1001', '0011']).
- `returns, costs`: Lists of integers representing the expected return and purchase cost of each stock.
- `budget`: An integer representing the maximum budget before penalties apply.

**Output for `generate_evolution`:** A list containing the newly generated binary strings after 8 generations.

**Sample Input:**

```
population = ['1101', '0110', '1001', '0011']
returns = [45, 35, 25, 15]
costs = [12, 18, 10, 6]
budget = 25
```

**Sample Output:**

```
['1001', '0111', '1100', '0010']
```

*(Note: The exact output will vary due to the randomness in truncation selection, single-point crossover, mutation, and 8 generations of evolution.)*